

# assertpy Cheat Sheet

Complete reference for `assert_that()` methods

## Equality & Identity

|                                    |                                 |
|------------------------------------|---------------------------------|
| <code>.is_equal_to(val)</code>     | Assert value equals val (==)    |
| <code>.is_not_equal_to(val)</code> | Assert value does not equal val |
| <code>.is_same_as(val)</code>      | Assert identity match (is)      |
| <code>.is_not_same_as(val)</code>  | Assert no identity match        |

## Truth / None

|                             |                          |
|-----------------------------|--------------------------|
| <code>.is_true()</code>     | Assert value is True     |
| <code>.is_false()</code>    | Assert value is False    |
| <code>.is_none()</code>     | Assert value is None     |
| <code>.is_not_none()</code> | Assert value is not None |

## Strings

|                                             |                                          |
|---------------------------------------------|------------------------------------------|
| <code>.contains('substring')</code>         | Assert string contains substring         |
| <code>.does_not_contain('substring')</code> | Assert string does not contain substring |
| <code>.starts_with('prefix')</code>         | Assert string starts with prefix         |
| <code>.ends_with('suffix')</code>           | Assert string ends with suffix           |
| <code>.matches(r'regex')</code>             | Assert string matches regex pattern      |
| <code>.is_empty()</code>                    | Assert string is empty                   |
| <code>.is_not_empty()</code>                | Assert string is not empty               |
| <code>.is_length(n)</code>                  | Assert string length equals n            |

## Numbers

|                                              |                                         |
|----------------------------------------------|-----------------------------------------|
| <code>.is_greater_than(n)</code>             | Assert value > n                        |
| <code>.is_greater_than_or_equal_to(n)</code> | Assert value >= n                       |
| <code>.is_less_than(n)</code>                | Assert value < n                        |
| <code>.is_less_than_or_equal_to(n)</code>    | Assert value <= n                       |
| <code>.is_between(low, high)</code>          | Assert low <= value <= high             |
| <code>.is_close_to(val, tolerance)</code>    | Assert value is within tolerance of val |
| <code>.is_zero()</code>                      | Assert value is 0                       |
| <code>.is_positive()</code>                  | Assert value > 0                        |
| <code>.is_negative()</code>                  | Assert value < 0                        |

## Lists / Collections

|                                               |                                             |
|-----------------------------------------------|---------------------------------------------|
| <code>.contains(item)</code>                  | Assert collection contains item             |
| <code>.contains_only(item1, item2)</code>     | Assert collection contains only these items |
| <code>.contains_sequence(item1, item2)</code> | Assert items appear in order                |
| <code>.does_not_contain(item)</code>          | Assert collection does not contain item     |
| <code>.is_empty()</code>                      | Assert collection is empty                  |
| <code>.is_not_empty()</code>                  | Assert collection is not empty              |
| <code>.is_length(n)</code>                    | Assert collection length equals n           |
| <code>.has_size(n)</code>                     | Alias for <code>is_length(n)</code>         |
| <code>.is_subset_of(other)</code>             | Assert collection is a subset of other      |

## Dicts

|                                           |                                |
|-------------------------------------------|--------------------------------|
| <code>.contains_key('key')</code>         | Assert dict has key            |
| <code>.does_not_contain_key('key')</code> | Assert dict does not have key  |
| <code>.contains_value('val')</code>       | Assert dict has value          |
| <code>.contains_entry(key='val')</code>   | Assert dict has key-value pair |

## Type Checking

|                                             |                                       |
|---------------------------------------------|---------------------------------------|
| <code>.is_instance_of(SomeClass)</code>     | Assert value is instance of class     |
| <code>.is_not_instance_of(SomeClass)</code> | Assert value is not instance of class |
| <code>.is_type_of(type)</code>              | Assert exact type match               |

## Exceptions

|                                      |                                       |
|--------------------------------------|---------------------------------------|
| <code>.raises(ValueError)</code>     | Assert callable raises exception      |
| <code>.when_called_with(args)</code> | Trigger the call (chain after raises) |

*# assert\_that(fn).raises(ValueError).when\_called\_with(bad\_arg)*

## Chaining

```
assert_that(response.json()["email"]) \
    .is_not_none() \
    .is_not_empty() \
    .ends_with("@pytest.com")
```